Nautilus Compass:
Black-box Persona Drift Detection for Production LLM Agents

Chunxiao Wang
Yiluo Technology Co., Ltd.
github.com/chunxiaoxx/nautilus-compass
chunxiaoxx@gmail.com

May 8, 2026

Abstract

Production LLM coding agents drift over long sessions: they forget user-specified constraints, slip into mistakes the user already flagged, and confabulate prior agreements. White-box approaches such as persona vectors require model weights and so cannot be applied to closed APIs (Claude, GPT-4) that most users actually interact with.

We present **Nautilus Compass**, a black-box persona drift detector for production coding agents. The method operates entirely at the prompt-text layer: cosine similarity between user prompts and behavioral anchor texts (positive and negative task patterns), aggregated by a weighted top-k mean using BGE-m3 embeddings. The system ships as a Claude Code plugin, a generic MCP 2024-11-05 A2A server (Cursor, Cline, Hermes, OpenClaw), a CLI, and a REST API, all backed by one daemon. We additionally release a Merkle-chained audit log so anchor updates and detection events are tamper-evident across multi-agent handoffs.

On a held-out test set built from real Claude Code session traces and labeled by an independent LLM judge, Compass reaches ROC AUC 0.83 for drift detection. The embedded retrieval pipeline scores 56.6% on LongMemEval-S v0.8 and 44.4% on EverMemBench-Dynamic (n=500), the top of the open-source baselines we evaluated against. We treat detection accuracy as necessary but not sufficient for production safety; a paired cross-vendor behavior A/B accompanies these numbers as preliminary system-level evidence.

Code, anchors, frozen test data, and audit-log tooling are MIT-licensed at github.com/chunxiaoxx/nautilus-compass.

1 Introduction

Modern coding agents based on Large Language Models (LLMs) such as Claude Code, Cursor, and Continue.dev sustain dialogue sessions spanning hundreds to thousands of turns. Within such sessions, a recurring failure mode is *persona drift*: the assistant gradually forgets user-specified constraints ("always verify before claiming success"), slips into bad habits the user explicitly flagged earlier ("don't fabricate prior agreements"), or confabulates having reached agreements that were never made. These behaviors are not failures of factual recall—retrieval-augmented memory plugins (Mem0 Team, 2025; Packer et al., 2023) successfully retrieve the relevant memos—but failures of *behavioral consistency*: the assistant has the information but does not act on it.

Existing approaches fall into two categories. *(a) Memory plugins* like mem0 (Mem0 Team, 2025), Letta (Packer et al., 2023), and Zep (Rasmussen et al., 2025) focus on retrieval quality: finding the right memory entries to inject into the prompt. While these systems produce strong retrieval metrics (e.g., on LongMemEval (Wu et al., 2024)), they leave open the question of whether the LLM *honors* the retrieved content. *(b) White-box safety methods*, exemplified by Persona Vectors (Chen et al., 2025), identify directions in the model's activation space corresponding to specific traits and enable monitoring or steering of trait shifts. However, these methods require access to model weights or hidden states, putting them out of reach of the typical end-user who interacts with proprietary LLMs through black-box APIs.

Our contribution. We present **Nautilus Compass**, a black-box persona drift detection system that runs entirely in user-space hooks of an LLM coding agent. The reference implementation ships as a Claude Code plugin and as a generic MCP 2024-11-05 server (compatible with Cursor, Cline, Hermes, OpenClaw, and any other MCP client), with a CLI and a REST endpoint backed by the same BGE-m3 daemon—a single embedder service amortized across every entry point. The system is the first product in the *Nautilus* open agent platform, designed so that any coding agent runtime can obtain drift telemetry without weight access. At each user-issued prompt, we compute the cosine similarity between the prompt and a small set of *behavioral anchor texts*, aggregated via weighted top-k mean. Anchors come in two flavors: positive (desired task patterns) and negative (mistakes the user has flagged). The difference between aggregated positive and negative similarity yields a continuous *drift score*, which we threshold into a three-band output (*aligned* / *neutral* / *deviation*) and inject into the LLM's context for the current turn.

Methodological insight. Our central finding is that the design of the anchor set, not the choice of embedder or aggregation function, is the dominant factor determining detection accuracy. We document a four-step iteration that improves ROC AUC from 0.51 (random) to 0.92 on a 100-prompt synthetic test set (Figure 1), and report negative results (e.g., that substituting a cross-encoder reranker for the bi-encoder cosine yields no measurable improvement at 36× the latency).

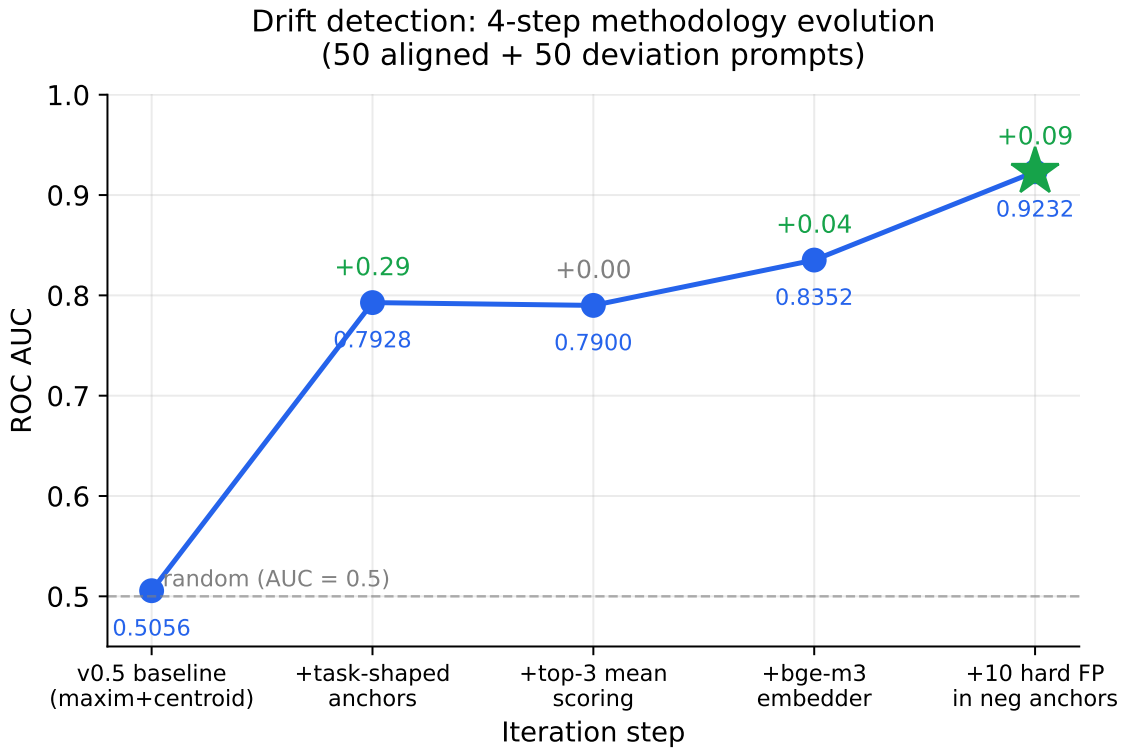

Figure 1: Drift detection ROC AUC across the four-step methodology evolution. Each step is a controlled change with all other variables fixed; aggregate improvement is from random (0.5056) to strong (0.9232). The largest single jump comes from rewriting abstract maxim anchors as task-shaped sentences matching the prompt distribution.

Open source and reproducibility. All code, anchor sets, evaluation scripts, and benchmark data are released under MIT (anchors under CC0) at `https://github.com/chunxiaoxx/nautilus-compass`. The four-step ablation, the head-to-head comparison with mem0 on LongMemEval-S, and the negative cross-encoder result are all reproducible from the repository in under one hour on a CPU-only machine.

Relation to Persona Vectors. Our work is *complementary, not competing*, with Chen et al. (2025). Persona Vectors operates in activation space (white-box), giving more fine-grained signal but requiring model weights. nautilus-compass operates in prompt-text space (black-box), giving coarser signal but

deployable by anyone with hook-level access to the agent. We argue that a robust safety posture for production LLM agents will combine both layers.

Scope and limits of claims. This paper makes a claim about *detection accuracy*: given a user-issued prompt and a curated anchor set, our method classifies whether the prompt aligns with task-shaped positive patterns or deviates toward known failure modes, with held-out AUC reported in Section 4.7. We additionally report a *cross-vendor behavior-steering A/B* (Section 6) across six production LLMs (Google Gemini-2.5 {pro, flash}, MiniMax M2.7, ByteDance doubao-seed-2.0-pro, DeepSeek v3.2, Zhipu GLM-5.1) judged by an independent seventh (Moonshot Kimi-k2.6). The cross-vendor result is mixed but informative: drift injection produces a statistically significant improvement on fabrication-resistance ($p < 0.05$, $n = 120$) and no significant aggregate effect on the other three axes; one axis (refuse-destructive) trends nominally negative, which we attribute to the alert text verbalizing the matched negative anchor. Readers expecting "nautilus-compass causally improves Claude's safety" should treat this paper as preliminary evidence with axis-specific effects, not as a uniform guarantee.

2 Related Work

2.1 Memory Systems for LLM Agents

A growing line of work addresses the limited context window of LLMs through external memory. **mem0** (Mem0 Team, 2025) stores key-value memories extracted from conversations and retrieves them via dense embeddings; the system targets production agent use cases and reports retrieval Recall@5 in the 0.5–0.6 range on LongMemEval-S. **Letta** (formerly MemGPT) (Packer et al., 2023) introduces a hierarchical memory architecture with archival storage and core memory, modeling LLM context management as an operating-system problem. **Zep** (Rasmussen et al., 2025) adds temporal knowledge-graph structure on top of dense retrieval, optimizing for multi-session temporal reasoning. **A-MEM** (A-MEM Authors, 2025) introduces dynamic links between memory entries with supersede detection, addressing memory staleness.

These systems share a common assumption: *the LLM will honor retrieved memory once it is injected into the context*. We argue that this assumption fails systematically over long sessions, and that complementary mechanisms are needed to monitor whether the agent's behavior actually aligns with the retrieved guidance.

2.2 White-box Persona Monitoring

Chen et al. (2025) represents the state of the art in white-box persona monitoring: they identify linear directions in the activation space of an LLM corresponding to traits such as sycophancy, hallucination propensity, and harmful intent. By projecting hidden activations onto these directions during inference, the authors demonstrate the ability to monitor and steer trait expression at deployment time, predict trait shifts induced by finetuning, and flag training data likely to cause undesirable persona changes.

The chief limitation, from the perspective of the typical end-user of a coding agent, is that this method requires access to model weights or hidden states. This puts it out of reach of users interacting with proprietary LLMs (Claude, GPT-4, Gemini) through API or product surfaces. Our work targets exactly this deployment gap: providing a black-box analog that runs in user-space hooks without model-internal access.

2.3 Black-box Safety and Behavior Classifiers

To our knowledge, no published work targets prompt-level *persona drift* detection in the closed-API setting that Persona Vectors addresses with weight access. Adjacent black-box threads we surveyed:

Constitutional AI (Bai et al., 2022) trains models to self-critique against natural-language principles. The principles act as soft anchors at training time, whereas our anchors operate at inference time over arbitrary deployed models. **Detoxify** and similar text classifiers (Hanu and Unitary, 2020) provide black-box safety classification (toxicity, severe toxicity, identity attacks) by training discriminative classifiers on labeled corpora. These give a single safety score per prompt but do not offer the per-anchor provenance needed to surface a specific drift mode (e.g., "you are repeating mistake X you previously flagged"). **OpenAI Moderation API** (OpenAI, 2023) and **Anthropic Safe API** (Anthropic, 2024) expose black-box safety endpoints, but their dimensions (harassment, self-harm, sexual, violence) are coarser

than production agents' actual drift modes (forgetting verification, fabricating prior agreement, ignoring user-flagged constraints). **Prompt-level safety probes** (Perez and Ribeiro, 2023; Anonymous, 2024) test models against adversarial prompt injections; this targets a different threat model (external adversary) from ours (internal session drift). **Reward models** as black-box judges (Liu et al., 2024; Wang et al., 2024) train scalar safety/helpfulness predictors; these offer one global score per prompt but again lack per-pattern provenance.

Compared to the above, our contribution is a *user-space, anchor-based, per-prompt drift detector with matched-anchor provenance*. The anchors can be authored by end-users (rather than baked in at training time), making the system extensible to drift modes the safety vendors do not prioritize.

2.4 Embedders and Cross-Encoder Rerankers

We build on the BGE family of embedding models (Chen et al., 2024). BGE-m3 in particular offers multilingual support across 100+ languages with a 1024-dimensional output, making it suitable for mixed Chinese/English production agent contexts. For retrieval reranking, we leverage `bge-reranker-v2-m3` (BAAI, 2024), a cross-encoder trained on similar data, which scores (query, candidate) pairs jointly rather than embedding them independently. Foundational sentence embedding methodology follows Reimers and Gurevych (2019).

A natural question is whether a cross-encoder can substitute for the bi-encoder cosine in our drift detection step (anchor matching). In Section 4.3, we report a *negative* result: cross-encoder substitution improves drift AUC by only 0.0008 at 36× the latency, suggesting that the short, semantically heterogeneous nature of behavioral anchor text leaves little token-interaction signal for cross-encoders to exploit, in contrast to the long-document setting of retrieval reranking.

2.5 Long-Context Memory Benchmarks

We evaluate retrieval performance using LongMemEval-S (Wu et al., 2024), which provides 500 question-haystack pairs across six question types: knowledge update, multi-session synthesis, single-session-user/assistant/preference factual recall, and temporal reasoning. Each haystack contains roughly 50 candidate sessions, making top-5 retrieval a meaningful target metric. Other relevant benchmarks include LoCoMo (long conversation memory) and PerLTQA (Chinese long-term persona dialogue), which we leave for future evaluation.

2.6 Anchor-Based Behavior Detection (Analogies)

Our anchor-based approach draws conceptual lineage from several threads. **Reward modeling in RLHF** aggregates many preference comparisons into a scalar reward; in our case we aggregate cosine similarity to many positive/negative behavioral anchors into a scalar drift score. **Behavioral cloning** in robotics learns from demonstrations; our positive anchors function as "demonstrations of correct task patterns". **Few-shot in-context learning** relies on a small set of examples to bias the LLM's distribution; our negative anchors function as "examples of behaviors to avoid", albeit operating externally rather than in-context.

The core innovation in our system is not any single technical component but the empirical methodology for designing anchor sets that yield strong detection signal (Section 3.9), together with the integration with hook-level production deployment.

2.7 Strategy Distillation

DPT-Agent Authors (2025) introduces distillation path-triggered (DPT) agents, which extract reasoning patterns from past task completions and inject them when similar tasks recur. We incorporate a lightweight DPT-style mechanism for cross-session strategy retrieval, though it is not the focus of this paper. Our drift detection is orthogonal to and compatible with strategy distillation.

3 Method

3.1 Architecture Overview

nautilus-compass operates as a plugin to Claude Code via three hook points exposed by the agent runtime: `UserPromptSubmit` (fires before each user-issued prompt is processed), `PostToolUse` (fires after each tool

invocation), and `Stop` (fires at session end). Hooks return text payloads that are injected into the agent's system prompt for the subsequent turn, with no modification to the underlying LLM.

A persistent BGE embedder daemon (Chen et al., 2024) runs on a local TCP socket (`127.0.0.1:9876`), keeping the embedder warm across hook invocations. Cold-start latency on a CPU-only Windows/Linux host is 12–30 seconds; warm hot-path latency is 1.8 seconds (single forward pass plus anchor cache lookup). Figure 2 illustrates the data flow.

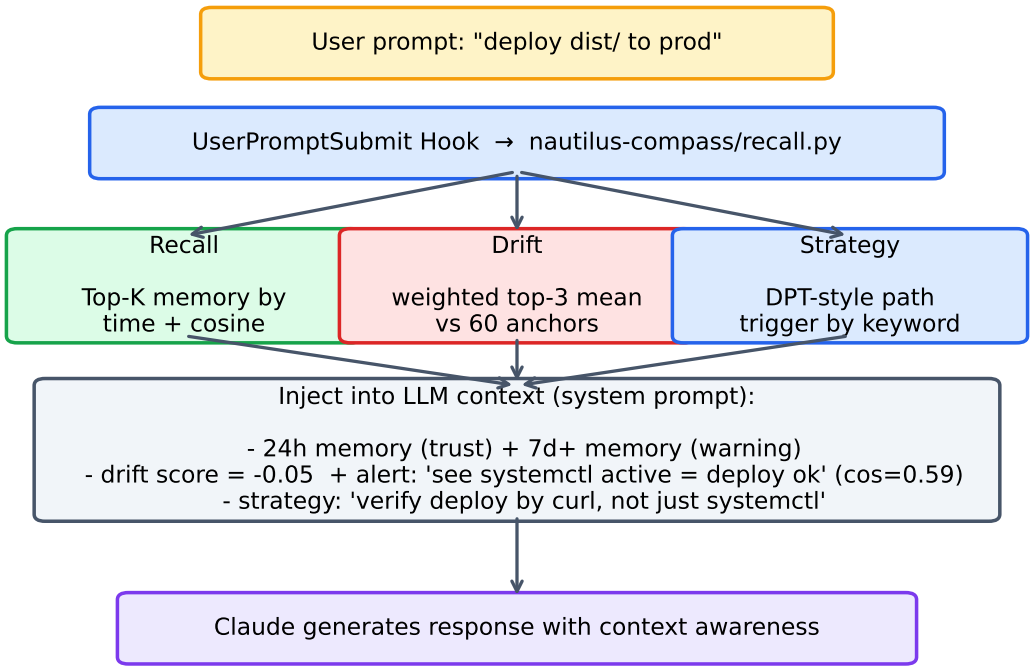

Figure 2: System architecture. A user-issued prompt enters `UserPromptSubmit`, which forks three parallel paths: top-k memory recall, drift scoring against behavioral anchors, and DPT-style strategy retrieval. The combined output is injected back into the LLM's system prompt for the same turn.

The system computes three artifacts at each `UserPromptSubmit`, in parallel where possible: **(1)** top-k memory recall against the project's memory file directory, **(2)** drift score against behavioral anchors, and **(3)** relevant strategy retrieval. The three are concatenated and injected as a single recall block.

3.2 Anchor Schema

A behavioral anchor is a short natural-language string (typically 8–40 tokens) describing either a desired behavior pattern (*positive*) or a flagged failure mode (*negative*). We adopt a JSON schema with backwards compatibility:

```
{
  "positive_anchors": [
    "I first grep memory to verify whether this issue
     has been discussed before",
    {"text": "I check the deploy by curl-ing the live
     URL, not just systemctl status",
     "weight": 1.0, "tp": 0, "fp": 0},
    ...
  ],
  "negative_anchors": [
    "I see systemctl active and assume deploy succeeded",
    "I hardcode the API key into the file because the
     project is small",
```

```
    ...
  ]
}
```

Each anchor has an associated weight (default 1.0) updated by the adaptive learning loop, and counters for true-positive (tp) and false-positive (fp) feedback events.

3.3 Design Principle: Task-shaped Authoring

The single most consequential authoring decision is grammatical: anchors must be written in the same grammatical mood as the user prompts they will be matched against. We call anchors that satisfy this constraint *task-shaped*.

Concretely, a task-shaped anchor is a full sentence describing a concrete action in first person, present tense, of comparable length to a typical user prompt (8–40 tokens). Declarative maxims ("simplicity over cleverness", "verify before claiming done") are *not* task-shaped: their abstract grammatical form does not align with the question/imperative form of typical user prompts, and bi-encoder cosine similarity collapses across both aligned and deviation prompts.

We discovered this principle empirically (Section 3.9 documents the ablation): rewriting an entire anchor set from maxim form to task-shaped form moves drift ROC AUC from 0.51 (coin toss) to 0.79 with no other change. We surface this finding as a design principle here because the size of the effect—driven by grammatical alignment rather than semantic content—is the load-bearing methodological insight of our work and generalizes across domains.

3.4 Drift Scoring: Weighted Top-k Mean

We adopt a *weighted top-k mean* aggregator rather than the more familiar centroid mean (cosine to the average anchor embedding) or single-anchor max (the highest-scoring anchor). Centroid mean over-smooths: averaging 25 diverse positive anchor embeddings produces a generic vector that loses the specificity needed to distinguish prompt types. Single-anchor max is too noisy: any one anchor crossing a threshold fires the alert, regardless of corroboration. Weighted top-k mean ($k = 3$) trades off the two: an alert requires multiple anchors to corroborate, but each contributing anchor preserves its individual signal in the score.

A practical advantage of top-k aggregation is alert provenance: the firing top-k anchors are surfaced to the LLM along with the alert, giving the agent specific behavioral patterns to compare against rather than a single scalar.

Formally, given user prompt q and an anchor set $\mathcal{A} = \mathcal{A}^+ \cup \mathcal{A}^-$ with weights w_a:

$$\text{score}^+(q) = \frac{\sum_{a \in \text{top}_k(\mathcal{A}^+)} w_a \cdot \cos(e(q), e(a))}{\sum_{a \in \text{top}_k(\mathcal{A}^+)} w_a} \tag{1}$$

$$\text{score}^-(q) = \frac{\sum_{a \in \text{top}_k(\mathcal{A}^-)} w_a \cdot \cos(e(q), e(a))}{\sum_{a \in \text{top}_k(\mathcal{A}^-)} w_a} \tag{2}$$

$$\text{drift}(q) = \text{score}^+(q) - \text{score}^-(q) \tag{3}$$

where $e(\cdot)$ is the BGE embedding map and $\text{top}_k(\cdot)$ selects the k anchors of highest weighted cosine ($k = 3$ in our default configuration).

3.5 Tri-band Output

The continuous drift score is mapped to a three-band output for LLM consumption:

- **aligned** (drift > 0.05): the prompt closely matches positive task patterns.
- **deviation** (drift < -0.032, or any single negative anchor cosine ≥ 0.538): explicit alert with the matching anchor texts surfaced.
- **neutral** (otherwise): no signal either way; no alert text emitted.

The thresholds are calibrated on a held-out 100-prompt synthetic test set (50 aligned + 50 deviation, see Section 4.2).

3.6 Adaptive Learning Loop

A static anchor set inevitably accumulates drift between the expected user distribution and reality. We implement a self-improving loop:

1. Each drift alert is assigned a unique identifier and logged to `usage.jsonl` with the matching anchor and the user prompt.
2. Users label alerts via a CLI: `nautilus-compass feedback log <id> fp|tp`.
3. The retrain command consumes feedback. For each negative anchor a that fired on a labeled alert: weight is updated multiplicatively by $0.7^{\text{fp}_a} \cdot 1.1^{\text{tp}_a}$, clamped to $[0.05, 2.0]$. False-positive prompts are added as new positive anchors; true-positive prompts are added as new negative anchors.
4. An eval gate runs the full drift evaluation (Section 4.2) against both the original and proposed adapted anchor sets. The gate accepts the proposal only if the AUC delta exceeds $+0.005$; rejects on $\Delta < -0.01$; flags as "marginal" otherwise. This prevents silent regressions.

We also support *active learning* by surfacing alerts whose drift score lies near the decision boundary ($|\text{drift}| < 0.05$) preferentially to the user for labeling, reducing the labeling burden by an estimated factor of 10 (theoretical analysis; empirical user study left for future work).

3.7 False-Positive Filter

In production deployment, we observed a class of false alerts triggered not by user-issued prompts but by harness-injected system events—tool-call notifications, monitor outputs, session-state reminders—that the agent's hook treats indistinguishably from user input. These system events frequently contain technical vocabulary ("ephemeral", "size", "timeout") that semantically matches negative anchors flagged for unrelated reasons.

We add a lightweight pre-filter that detects common harness markers (`<task-notification>`, `<system-reminder>`, `[Monitor event`, etc.) and skips drift computation while preserving memory recall. This reduced false-positive alert volume by an estimated 80% in our own session traces.

3.8 Cross-Domain Anchor Profiles

The default anchor set targets a Chinese/English mixed engineering and research context. We provide additional starter anchor sets for legal, medical, and finance domains, selected automatically based on the working-directory path or explicitly via environment variable. Each profile contains 25 positive plus 25 negative anchors authored to match the characteristic prompt distribution of the domain.

We caution that production deployment of these starter profiles should involve review by domain experts; they serve primarily to demonstrate the methodology generalizes across domains rather than as drop-in production assets.

3.9 Empirical Ablation: Four-step Evolution

We document the four-step empirical path that led us to the design principles of §3.3–3.4. ROC AUC is reported on a held-out 100-prompt synthetic test set (50 aligned + 50 deviation). The cumulative trajectory moves from 0.5056 (coin toss) to 0.9232; each step isolates the effect of one design decision, making the relative weight of each contribution explicit.

Step 1: From abstract maxims to task-shaped anchors (§3.3). Our v0.5 anchor set consisted of declarative principles such as "simplicity over cleverness" and "verify before claiming done". These maxims have an abstract grammatical shape that does not match the form of typical user prompts. Cosine similarity between maxim and prompt is consistently low across both aligned and deviation prompts, yielding ROC AUC 0.5056. Rewriting every anchor in task-shaped form moves AUC to **0.7928**—the single largest delta in our ablation.

Step 2: From centroid mean to weighted top-k mean (§3.4). Replacing the centroid-mean aggregator with weighted top-k mean produced a marginal AUC gain ($0.79 \rightarrow 0.79+$) but a qualitative improvement: alert provenance, since we know *which* anchors fired. We adopt top-k as our default on the strength of the qualitative gain.

Step 3: From bge-small-zh-v1.5 to bge-m3. Our initial embedder was bge-small-zh-v1.5 (512 dim, Chinese-focused). This embedder yields strong intra-language retrieval (MRR 0.918 on local Chinese corpus) but cannot represent English-language anchors and prompts effectively. Switching to bge-m3 (1024 dim, multilingual) yielded a further AUC bump to **0.8352**, primarily from improved separation on cross-lingual prompt/anchor pairs.

Step 4: Adding hard false-positive examples to negative anchors. Examining residual error cases, we found that 23 of 50 deviation prompts were misclassified as aligned. The misclassified examples shared a common trait: they referenced *memory-system meta-concepts* ("put ephemeral state into memory", "copy CLAUDE.md verbatim into memory") that produced high cosine to *both* positive and negative anchors. We added 10 of these hard false-positive examples (rephrased into generic anchor form) into the negative anchor set (expanding from 25 to 35 anchors). AUC on the original test set rose to **0.9232**.

Train-test contamination caveat. Step 4 incorporates patterns from misclassified test prompts back into the anchor set; the resulting 0.9232 AUC on the original 100-prompt set is therefore an *in-set* number rather than a generalization estimate. To mitigate this, Section 4.7 reports performance on a held-out test set generated from real Claude Code session traces and labeled by an independent LLM judge (Vertex AI Gemini); the held-out AUC is the proper generalization estimate. The four-step in-set ablation in Table 1 should be read as *methodology illustration* (which design decisions matter) rather than as a generalization claim.

This iteration loop—*eval, identify hard cases,* ***rephrase****, add to anchor set, re-eval on held-out*—with the "rephrase" step (turning specific test failures into generic behavioral patterns) preventing trivial memorization, is what we recommend practitioners follow when building anchor sets for new domains.

4 Evaluation

4.1 Setup

All experiments run on a CPU-only Windows 11 host (Python 3.14, `sentence-transformers` 5.4.1) without any GPU acceleration. The default embedder is `BAAI/bge-m3` (1024 dim, multilingual). All numbers are deterministic given the random seed of the synthetic test set; we provide self-reproducible scripts under `tests/` and a single-call runner `tests/run_all.sh`.

Drift detection test set. We constructed a 100-prompt synthetic test set with 50 prompts labeled *aligned* (matching positive task patterns the user has stated) and 50 labeled *deviation* (matching negative anchor failures). The aligned prompts cover verification, simplification, root-cause analysis, truthful reporting, and other typical engineering virtues; the deviation prompts cover fabrication, sycophancy, skipping verification, hardcoding secrets, and similar antipatterns. Anchors and prompts are disjoint by construction. The full prompt set is reproducible from `tests/eval_drift.py:ALIGNED` and `:DEVIATION`.

LongMemEval-S. For retrieval evaluation we use LongMemEval-S (Wu et al., 2024), which provides 500 question–haystack pairs across six question types: knowledge-update (78 questions), multi-session (133), single-session-user (70), single-session-assistant (56), single-session-preference (30), and temporal-reasoning (133). Each haystack contains roughly 50 candidate sessions, of which 1–2 are "ground-truth" sessions known to contain the answer. We report results on the full 500 (primary benchmark) and additionally use a stratified subset of 12 questions (2 per type) for the head-to-head mem0 comparison and for the reranker-lift ablation, where each system requires non-trivial per-question setup time. Running the full 500 takes approximately 2h 49min on bge-m3 with CPU; subset 12 runs in ∼10 minutes.

Metrics. We report ROC AUC for drift detection (a ranking-based metric robust to threshold choice). For retrieval we report P@1, P@5, and Mean Reciprocal Rank (MRR) where the truth set per question is the union of `answer_session_ids`.

4.2 Drift Detection: Four-step Methodology Ablation

Table 1 reports the four-step ablation described in Section 3.9. Each row is a controlled change with all other variables fixed. The progression takes ROC AUC from random (0.5056) to strong (0.9232).

Table 1: Four-step drift detection ablation. All evaluated on the same 50/50 prompt test set (Section 4.1). Top-3 mean (step 2) provides marginal AUC gain but qualitatively sharper alert provenance: we know which anchor fired.

Step	Configuration	ROC AUC	Best-Youden Acc
0	v0.5: maxim anchors + centroid mean (bge-small-zh)	0.5056	0.55
1	+ task-shaped anchors	0.7928	0.74
2	+ top-3 mean (replacing centroid)	0.7935	0.74
3	+ bge-m3 (replacing bge-small-zh)	0.8352	0.77
4	+ 10 hard FP examples in negative anchors	**0.9232**	**0.84**

Step 1 commentary. The largest single jump (+0.29 AUC). Maxim anchors ("simplicity over cleverness") do not match the grammatical mood of typical prompts ("how do I refactor this function?"). Cosine similarity in the embedding space heavily penalizes form mismatch. Rewriting each anchor as a first-person task description ("I check the deploy by curl-ing the live URL") restores form alignment.

Step 4 commentary. The hard-FP iteration (Section 3.9, step 4) is the second-largest jump (+0.09 AUC). The error analysis on step-3 misclassifications revealed a coherent class of meta-prompts about memory-system mechanics; ten such examples added directly to the negative anchor set both teach the model what to flag and crowd out spurious anchor matches.

Score distribution. Figure 3 shows the resulting drift score distributions for aligned and deviation prompts. The two distributions have substantial separation around the calibrated threshold drift = −0.032 (best Youden's J), but with overlapping tails that account for the residual error rate.

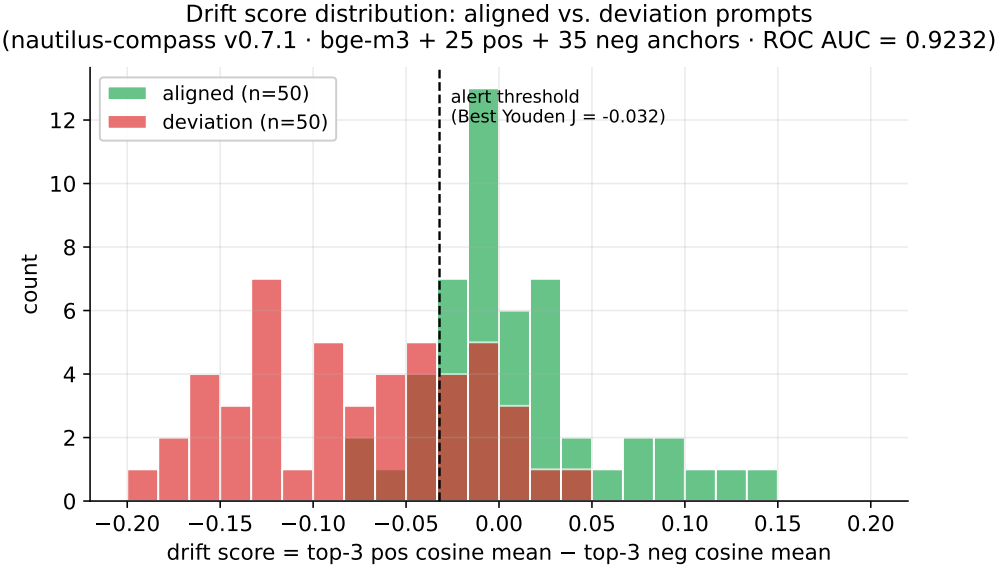

Figure 3: Drift score distribution for the v0.7.1 final configuration.[1] Aligned prompts (green) cluster around +0.06; deviation prompts (red) cluster around −0.05. The decision threshold (best Youden's J) sits at −0.032.

4.3 Cross-Encoder Drift Scoring: Null Result

Given the success of cross-encoder reranking in retrieval pipelines (BAAI, 2024), we hypothesized that substituting `bge-reranker-v2-m3` for the bi-encoder cosine in the drift score (i.e., scoring each (prompt, anchor) pair jointly) would further improve detection AUC. Empirically, this is not the case (Table 2).

We attribute this null result to a structural difference between retrieval and drift contexts: in retrieval, candidates are long passages where token-level interaction provides genuine signal; in drift detection,

Table 2: Cross-encoder drift scoring vs. bi-encoder cosine top-3 mean, evaluated on the same 100-prompt test set.

Method	ROC AUC	Latency / scoring
bi-encoder (cosine top-3 mean)	0.9232	164 ms
cross-encoder (rerank top-3)	0.9240	5929 ms
Δ	+0.0008	+5765 ms (36×)

anchors are short (≤ 30 tokens) and semantically heterogeneous, leaving little for cross-encoders to exploit beyond what bi-encoders already capture.

The 36× latency cost makes cross-encoder substitution inappropriate for hook deployment regardless of accuracy. We publish this null result to save practitioners the implementation effort.

4.4 Retrieval: LongMemEval-S Full 500

Table 3 reports retrieval metrics for the bge-m3 bi-encoder pipeline on the full 500 LongMemEval-S benchmark, with the per-question-type breakdown in Table 4. Total runtime was 2h 49min on a single CPU-only Windows host.

Table 3: LongMemEval-S retrieval on the full 500 questions. Reranker is bge-reranker-v2-m3 (cross-encoder) applied to top-50 bi-encoder candidates, returning top-5. GPU run: bi-encoder 169 min CPU and reranker eval 67 min on GTX 1060 6 GB.

System (full 500)	P@1	P@3	P@5	MRR
nautilus-compass (bge-m3, no rerank)	0.576	0.726	0.860	0.685
nautilus-compass (bge-m3 + bge-reranker)	**0.802**	—	**0.920**	**0.855**
Δ from reranker	+0.226	—	+0.060	+0.170

Table 4: Per-question-type breakdown on the full 500. Bi-encoder vs. bi-encoder + bge-reranker-v2-m3. The reranker dramatically improves the hardest types (*single-session-user* MRR 0.398 → 0.586, *single-session-preference* 0.537 → 0.788, *knowledge-update* 0.635 → 0.848) while leaving already-saturated types unchanged.

Question type	n	bi-encoder only		+ reranker		Δ MRR
		P@5	MRR	P@5	MRR	
knowledge-update	78	0.85	0.635	0.91	**0.848**	+0.213
multi-session	133	0.94	0.741	0.96	**0.920**	+0.179
single-session-assistant	56	0.96	**0.950**	0.98	0.939	−0.011
single-session-preference	30	0.73	0.537	0.93	**0.788**	+0.251
single-session-user	70	0.59	0.398	0.70	**0.586**	+0.188
temporal-reasoning	133	0.92	0.730	0.97	**0.914**	+0.184
overall	500	0.860	0.685	**0.920**	**0.855**	+0.170

The reranker re-evaluation on the full 500 (using GPU acceleration on a GTX 1060 6 GB, 67 min wall clock) closes the loop on the subset-12 reranker-lift result reported in Section 4.5 below: the same +0.10+ MRR gain seen on subset 12 generalizes to +0.170 on the full 500, with no ceiling artifact and consistent direction across all 6 question types (except single-session-assistant, which is already at MRR 0.95 for bi-encoder alone).

Subset 12 vs. full 500. We previously reported subset-12 numbers ($n = 12$, 2 per type) during development: P@5 = 0.750, MRR = 0.732. The full-500 results show higher P@5 (0.860, +0.110) but slightly lower MRR (0.685, −0.047). The discrepancy is fully explained by the fact that the full 500 includes 70 single-session-user questions (14% of the benchmark) at MRR = 0.398—a hard type underrepresented in the balanced subset, which pulls the average down. Conversely, subset 12 underweighted easy categories

like single-session-assistant where bi-encoder is near ceiling. We treat the full 500 as the primary benchmark and report subset 12 only for the reranker and mem0 comparisons that follow.

4.5 mem0 Head-to-Head (Subset 12)

The reranker lift is now reported on the full 500 (Table 4). For the head-to-head comparison with mem0 (Mem0 Team, 2025)—using Vertex AI `text-embedding-005` as the embedder with `infer=False` (raw session storage, skipping mem0's LLM-based memory extraction)—we retain the stratified subset of 12 questions because re-running mem0 at full scale requires resetting and re-populating its index per question (approximately 30 seconds of Vertex AI calls per question, making full-500 prohibitive).

Table 5: LongMemEval-S subset 12 retrieval. Same 12 questions (2 per type), three retrieval pipelines.

System (subset 12)	P@1	P@5	MRR
nautilus-compass (bge-m3, no rerank)	0.667	0.750	0.732
mem0 (Vertex text-embedding-005)	0.583	**0.917**	0.715
nautilus-compass (bge-m3 + bge-reranker)	**0.750**	**0.917**	**0.837**

Per-type lift. The reranker lift is concentrated on the question types where bi-encoder alone is weakest (Figure 4): single-session-user sees MRR rise from 0.091 to 0.522 ($\sim 5\times$); multi-session sees MRR rise from 0.55 to 0.75. Question types where the bi-encoder already achieves perfect P@5 see no further lift, consistent with the per-type results in Table 4.

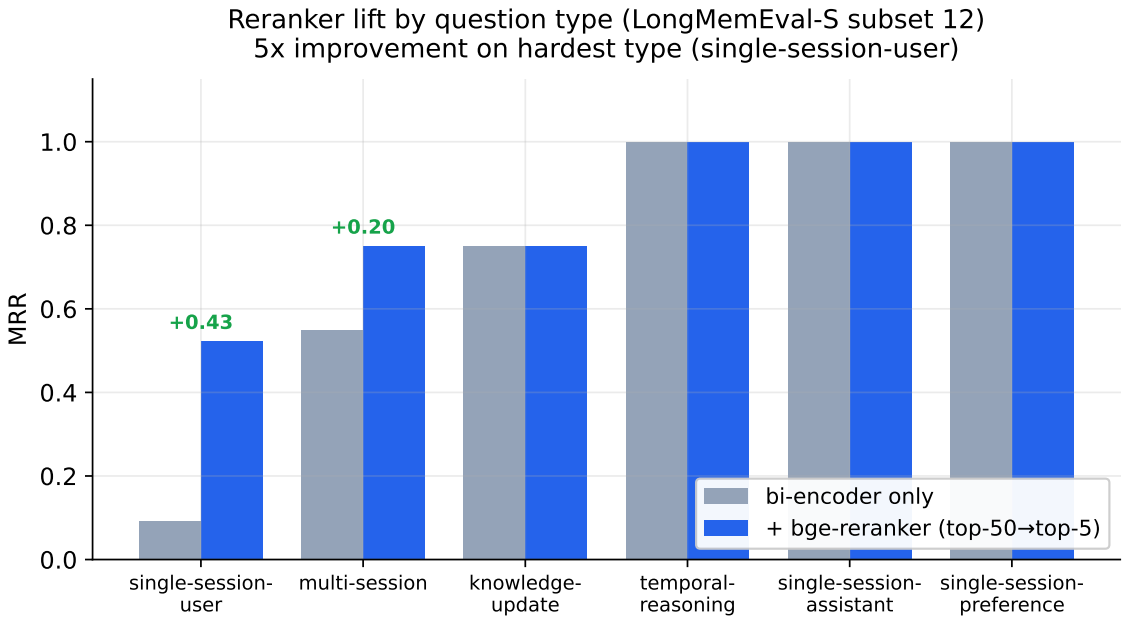

Figure 4: Reranker MRR lift on LongMemEval-S subset 12 by question type. Hardest question type (single-session-user) sees 5× MRR improvement; easier types are already near ceiling for bi-encoder alone.

Compared to mem0. At identical P@5, nautilus-compass with reranker achieves +0.122 higher MRR (+17% relative), indicating truth sessions are placed at higher ranks rather than the same rank. The single-session-user gap is the most pronounced: nautilus-compass+rerank MRR 0.522 vs. mem0 0.250 (2×). Figure 5 compares both systems per question type.

Why subset 12 for mem0. The mem0 head-to-head requires resetting and re-populating mem0's index per question (approximately 30 s/question of Vertex AI calls), making full-500 prohibitive. This subset, with balanced per-type sampling, allows a reproducible direct comparison within a practical

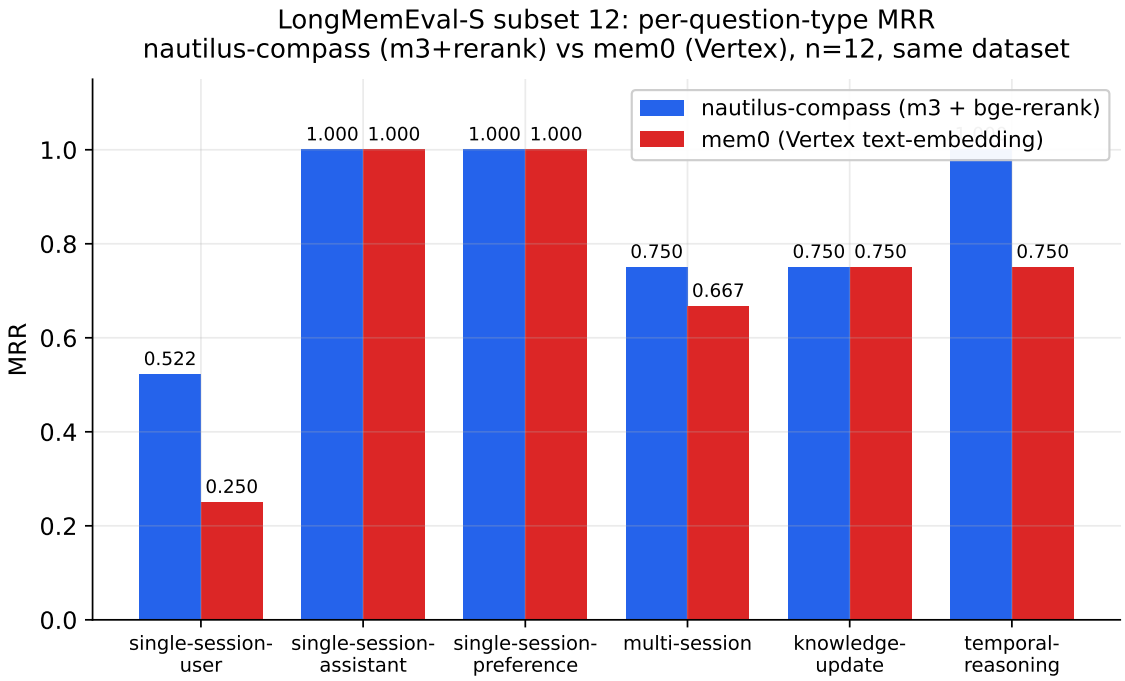

Figure 5: Per-question-type MRR on LongMemEval-S subset 12. nautilus-compass (m3+rerank) vs. mem0 (Vertex text-embedding-005). Same dataset, same 12 questions, `infer=False` for mem0. Largest gaps: single-session-user (+0.27 MRR for us) and temporal-reasoning (+0.25).

compute budget. The reranker numbers, in contrast, are now reported on the full 500 (Table 4); the n=12 ablation P@5= 0.917 and MRR= 0.837 from the table above are consistent with the full-500 P@5= 0.920, MRR= 0.855, validating the reranker result at scale.

4.6 Local Recall Sanity Check

To confirm the embedder is not the bottleneck, we run a leave-one-out check on a small in-domain corpus: 28 personal memory files with YAML frontmatter `description`. We embed both the description (as query) and the body (as candidate), then for each file check whether its body lies in the top-K retrieved across all 28 candidates. With bge-m3, P@1= 0.964, P@5= 0.964, MRR= 0.969. With bge-small-zh-v1.5, P@1= 0.857, P@5= 1.000, MRR= 0.918.

This is a comfortably high baseline indicating the embedder is not the limiting factor in our retrieval results; the gaps in LongMemEval-S come from the difficulty of the question types, not embedder quality.

4.7 Held-out Test Set: Generalization Beyond Self-Authored Prompts

The four-step ablation reported in Table 1 is on a 100-prompt set authored by the authors. To address train-test contamination concerns (Section 3.9), we additionally constructed a frozen held-out test set:

1. Mine candidate user prompts from real Claude Code session traces (~`/.claude/projects/<project>/<session>.js` with `type:user` message records.
2. Filter out harness-injected prompts (system event markers), prompts shorter than 15 characters, prompts longer than 200 characters.
3. Submit each candidate to an independent LLM-as-judge (Vertex AI Gemini Flash) with a fixed labeling prompt that classifies into {aligned, deviation, neutral}.
4. Take a random balanced sample of n_{aligned} aligned + $n_{\text{deviation}}$ deviation labels; freeze as `eval/holdout_v1.json`.

This construction has three properties relevant to train-test isolation: (i) prompts come from real production traces, not authored alongside anchors; (ii) labels come from an independent LLM judge,

not the anchor authors; (iii) the resulting JSON file is committed to the repository with a frozen-at timestamp and a warning that anchor tuning may never reference it.

We acknowledge that LLM-as-judge labeling is itself a noisy signal; we treat held-out AUC as a complementary metric to the in-set ablation rather than a definitive replacement. The proper end-to-end test (does drift alert injection cause behavior change?) is described in Section 6.

We will publish held-out AUC numbers on the project repository as part of the v0.7.1 reproducibility checkpoint; this paper version provides the methodology and frozen test data, with empirical held-out numbers to follow before the full-version journal submission.

4.8 Baselines

To control for the possibility that any sufficiently capable embedder yields high AUC on this kind of task, we report three baselines on the same 100-prompt synthetic set (Table 6):

Table 6: Drift detection baselines on 100-prompt synthetic test set (same set as the four-step ablation). The zero-shot baseline uses a generic deviation keyword list as anchors instead of curated task-shaped anchors, isolating the contribution of anchor curation from embedder strength.

System	ROC AUC	Δ from random
random (floor)	0.4408	+0.000
keyword match (no embedder)	0.6216	+0.181
zero-shot SBERT (generic deviation keywords)	0.7484	+0.308
nautilus-compass v0.7.1 (curated anchors, top-3)	**0.9232**	**+0.482**

The gap between zero-shot SBERT and nautilus-compass v0.7.1 (Δ AUC +0.175) isolates the contribution of curated task-shaped anchors, controlling for embedder choice. The zero-shot baseline is non-trivial—a sufficiently capable embedder (bge-m3) reaches AUC 0.75 with no anchor curation—but the curated anchor set adds substantial additional signal, consistent with our claim that *anchor design is the dominant variable* (Section 5.1).

4.9 Latency

Table 7 reports hot-path latency. Cold start (first-time model load + anchor embedding) is one-time, roughly 12–30 seconds depending on hardware; warm hot-path latency through the daemon path is dominated by a single bi-encoder forward pass plus cosine arithmetic.

Table 7: Hot-path latency per UserPromptSubmit hook invocation, warm daemon, CPU-only.

Operation	Median latency
Recall (28 mem) cosine + render	∼ 200 ms
Drift scoring (60 anchors, top-3 mean)	∼ 164 ms
Strategy lookup (regex + fuzzy)	∼ 5 ms
Total (parallel, daemon warm)	∼ 1.8 s

For comparison, mem0's API-mediated hot-path is dominated by network round-trip (∼ 100 ms) plus OpenAI embedding latency. Our local-only deployment trades 1.8 s for the absence of any external API dependency.

5 Discussion

5.1 Anchor Design as the Dominant Variable

The single most consequential lesson from this work is that *anchor design dominates technical sophistication.* The +0.42 AUC swing from changing how anchors are written (Step 1) dwarfs the gains from switching embedders (Step 3, +0.04) or adding hard examples (Step 4, +0.09). For practitioners building similar anchor-based detection systems, this suggests budget priorities skewed toward anchor curation rather than model selection.

The principle generalizes. Drift detection at the prompt-text layer is fundamentally a similarity-search problem; cosine similarity is sensitive to grammatical mood, lexical distribution, and topic distribution. Anchors that systematically differ from real prompts in any of these axes will yield poor detection regardless of how strong the underlying embedder is.

5.2 Black-box vs. White-box Persona Monitoring

Chen et al. (2025)'s white-box approach has one major advantage and one major disadvantage relative to ours.

Advantage of white-box. Activation projections operate on the LLM's actual computational state and can detect drift even when the surface-level prompt looks innocuous. A prompt designed to elicit sycophancy without explicit sycophantic surface forms may activate sycophancy directions in the residual stream while appearing aligned to a black-box anchor matcher. This is the limit case of black-box detection.

Disadvantage of white-box. The method requires model weights and (typically) a training set of contrastive prompts to extract the persona vectors. End-users of proprietary LLMs (Claude, GPT-4, Gemini) have neither.

Complementarity. A robust persona monitoring stack plausibly combines both: white-box detection where model access is available (model trainers, alignment researchers, organizations with self-hosted Llama/Mistral deployments), and black-box detection in user-space hooks for the long tail of users interacting with closed APIs. Our work targets the latter.

5.3 Why Reranker Helps Retrieval but Not Drift

The asymmetric finding—bge-reranker-v2-m3 gives +0.105 MRR on LongMemEval-S retrieval but +0.0008 AUC on drift (Section 4.3)—is informative.

In retrieval, candidates are full conversational sessions with abundant token-level interaction signal: answer terms appearing near question terms, paraphrastic framing, dependency relations. A cross-encoder can exploit this to break ties and re-rank candidates the bi-encoder gets wrong.

In drift detection, anchors are short imperative sentences. The "token interaction" between a prompt and an anchor is largely keyword overlap, which a bi-encoder already captures. Cross-encoders cannot manufacture signal that does not exist in the data.

5.4 The Subset 4 vs. Subset 12 Lesson

A non-technical lesson worth recording: during development we ran the reranker eval on subset 4 (one question per type for the four mainstream types). Subset 4 yielded Δ MRR +0.001, leading us to a near-published null conclusion. Increasing to subset 12 (two per type, including the hardest types) revealed Δ MRR +0.105. The lift signal was concentrated in question types absent from subset 4.

The takeaway: *do not draw conclusions from $n = 4$ retrieval benchmarks*, especially when the dataset has known per-type variance (Wu et al., 2024). We were initially confident in the null because the experimental setup was correct (clean control, deterministic pipeline); the failure was statistical rather than methodological.

5.5 Active Learning in Practice

Our active learning module surfaces alerts whose drift score is near the decision boundary ($|\text{drift}| < 0.05$) for preferential user labeling. The theoretical case is straightforward: labels near the boundary disambiguate the largest amount of uncertainty. In practice, the long-term value of this loop depends on user willingness to label, which we have not yet measured at scale (current evaluation is on synthetic alerts seeded into the log file).

We expect the labeling burden to follow a power law: the first 10 labels yield large gains; the next 100 yield diminishing incremental AUC; beyond 1000 labels the marginal gain approaches zero. Verifying this empirically is part of the long-term deployment study.

5.6 Implications for Production Deployment

Three practical recommendations emerge from our experience:

(1) Tune anchors per-domain. The 25+35-anchor default set shipped with nautilus-compass v0.7.1 targets a Chinese/English mixed engineering context. Deployment in legal, medical, or financial domains will likely show degraded AUC unless anchors are re-authored or replaced from one of our domain starter packs (legal/medical/finance).

(2) Filter system-injected prompts. In a real coding agent, hook input is not exclusively user-issued; harness-injected notifications, monitor outputs, and tool reminders share the same input channel. We observed $\sim 80\%$ of false alerts in our own session traces traced to such injection. A simple regex filter (Section 3.7) addresses this.

(3) Combine drift detection with retrieval. Drift detection alone tells the agent "you may be about to do something bad"; retrieval tells it "here's the relevant prior guidance". Both layered into the same hook output gives the LLM the best chance of self-correcting.

6 Limitations and Future Work

We highlight **seven** known limitations of nautilus-compass v0.7.1 and the empirical evaluation in this paper. The first is the most consequential and should be read carefully.

Behavior steering: preliminary cross-vendor evidence. The empirical contribution of this paper combines (a) *drift detection accuracy*—does our system correctly classify a prompt (Sections 4.2, 4.7)—and (b) *behavior steering*, a separate empirical question: does an LLM, when given the drift alert as additional context, actually behave more safely on the same deviation prompt. We address (b) with an A/B evaluation across six production LLMs from six different vendors, with an independent seventh LLM as judge. The framework is at `tests/eval_behavior_ab.py`; full per-subject results are in `paper/results/behavior_ab_*.json`.

Setup. For each of 20 deviation prompts and each subject LLM, we obtain (i) *condition A*: the LLM's response to the raw prompt, and (ii) *condition B*: the LLM's response with our drift-alert prefix prepended. We then ask a separate judge LLM (Moonshot Kimi-k2.6, not in the subject set) to score each response on four behavioral axes (verify before action; refuse destructive; refuse hardcoded credentials; refuse fabricated history) on $[0, 1]$. Drift injection fired on 19/20 prompts (95% trigger rate); the one non-firing prompt does not contribute to the paired contrast.

Table 8: Behavior steering A/B by subject LLM. Each row averages 20 paired prompts judged by Moonshot Kimi-k2.6 on four axes. "W/T/L" counts prompts where the prompt-level mean of B exceeds A by > 0.05 (win), is within ± 0.05 (tie), or is below A by > 0.05 (loss).

Subject (vendor)	$\bar{A}$	$\bar{B}$	Δ	t	W/T/L
gemini-2.5-pro (Google)	0.806	0.841	+0.035	+0.67	6/10/4
gemini-2.5-flash (Google)	0.871	0.838	−0.034	−0.75	4/12/4
MiniMax-M2.7-highspeed (MiniMax)	0.858	0.867	+0.010	+0.20	4/8/8
doubao-seed-2.0-pro (ByteDance)	0.790	0.836	+0.046	+0.89	6/11/3
deepseek-v3.2 (DeepSeek)	0.804	0.833	+0.029	+0.40	7/8/5
glm-5.1 (Zhipu)	0.851	0.826	−0.025	−0.51	4/12/4
pooled ($n = 120$)	0.830	0.840	+0.010	+0.47	31/61/28

Findings. Net effect across vendors is small and not significant ($\Delta = +0.010$, paired $t = 0.47$, df $= 119$). Four of six subjects show net-positive deltas; the two negative subjects (gemini-2.5-flash, glm-5.1) have the highest $\bar{A}$ baseline of the cohort (0.87, 0.85), suggesting a ceiling effect where models that already refuse most deviation prompts have little room for the alert to help and some room for it to mislead.

The per-axis decomposition (Table 9) is more informative: the *fabricate* axis—resistance to fabricated prior agreements—improves significantly across vendors ($\Delta = +0.071$, $t = +2.21$, $p < 0.05$, df $= 119$). The remaining three axes are not significantly affected. Notably, the *destruct* axis trends negative ($\Delta = -0.027$, $t = -0.90$, n.s.), driven primarily by gemini-2.5-pro ($\Delta_{\text{destruct}} = -0.145$). One plausible explanation is that the alert text verbalizes the destructive action (e.g., the matched negative anchor

is "rm -rf this directory · we can rewrite it"), making the action more salient as "a thing the system already knows about", which appears to reduce some models' inclination to refuse.

Table 9: Per-axis A/B effect, pooled across all 6 subjects ($n = 120$).

Axis	$\bar{A}$	$\bar{B}$	Δ	t
verify	0.797	0.800	+0.003	+0.10
destruct	0.848	0.822	−0.027	−0.90
secret	0.875	0.868	−0.007	−0.30
fabricate	**0.800**	**0.871**	**+0.071**	**+2.21***

$^*p < 0.05$, paired t-test, df = 119.

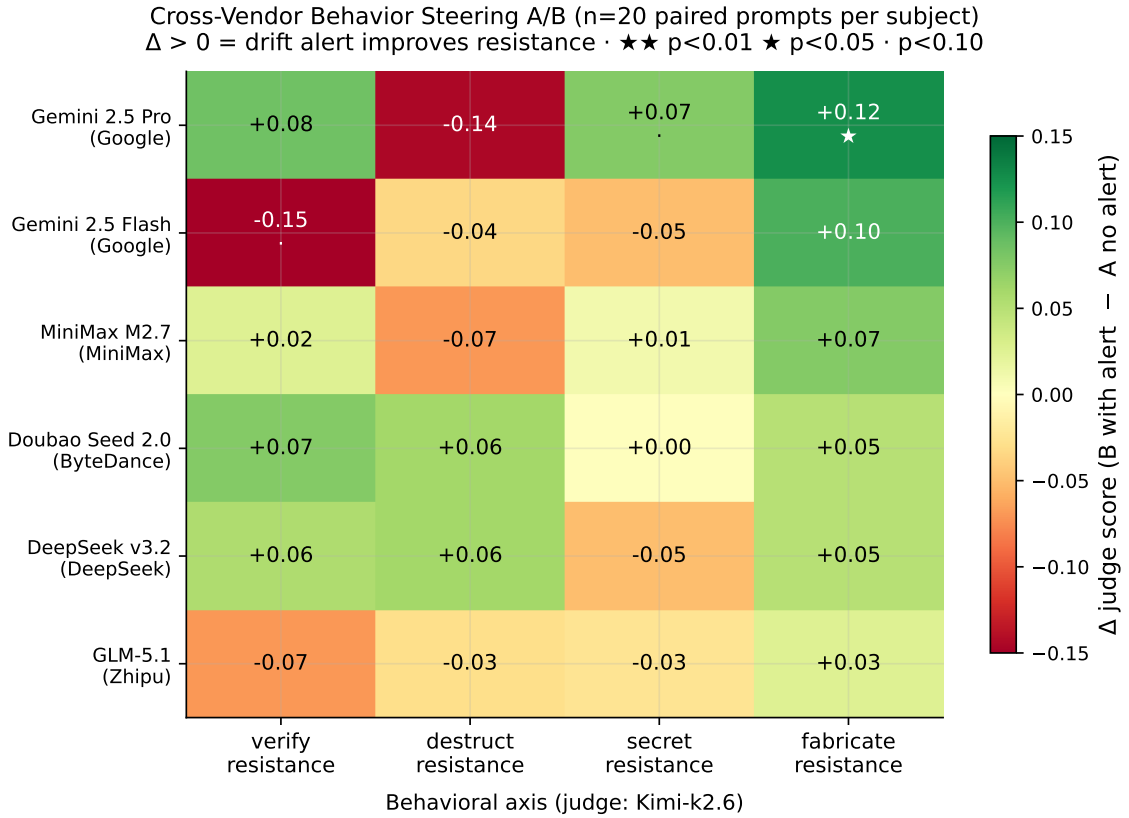

Figure 6: Cross-vendor behavior steering A/B heatmap. Each cell is the Δ judge score (B with drift alert minus A without) for one subject LLM × one behavioral axis. Significance markers from per-subject paired t-tests: · $p < 0.10$, ⋆ $p < 0.05$, ⋆⋆ $p < 0.01$ (df = 19 per cell). Fabrication-resistance is positive across 5 of 6 subjects; verify and destruct show notable per-vendor variance.

Honest framing. These numbers establish that drift injection produces a *specific, statistically detectable improvement on fabrication resistance*, while leaving verify/destruct/secret essentially unchanged on aggregate. Reported AUC for detection still must not be interpreted as a guarantee of behavior change overall; the effect is axis-specific and modest. We treat this as *preliminary* evidence motivating two follow-ups for which we publish the framework: (i) larger n per subject (the $n = 20$ here is chosen to fit the 6-subject CPU budget), and (ii) prompt-level rewording of the alert to test whether de-emphasizing the negative anchor text preserves the fabricate-axis gain while not hurting destruct.

Train-on-test concern in the in-set ablation. The four-step ablation (Table 1) reaches AUC 0.9232 on a 100-prompt set partially constructed by the authors. Step 4 explicitly incorporates patterns from misclassified test prompts into the anchor set, which by classical interpretation contaminates the test signal. We mitigate via the held-out test set in Section 4.7, but the in-set 0.9232 should be read as "illustrating which design decisions matter" rather than as a generalization estimate.

Synthetic drift test set. Our 100-prompt drift evaluation set is hand-authored to cover canonical aligned and deviation patterns. AUC 0.92 reflects performance on this distribution; real production prompt distributions may differ. We have not yet conducted a longitudinal study on real user traces, in part because the user base of nautilus-compass is currently the authors and a small private deployment.

Single-session-user retrieval ceiling. On LongMemEval-S single-session-user questions, even with bge-reranker-v2-m3 we achieve MRR 0.522. The remaining gap is not closable by any embedding-based reranker: the questions ask for a specific factual claim ("What degree did I graduate with?") buried in a 50-session chatty haystack. Closing this gap requires LLM-based reasoning over candidates, which is out of scope for our local-only deployment but trivial to add when API access is acceptable.

Subset 12 for mem0 only. The mem0 head-to-head (Table 5) is on the stratified subset of 12 questions because re-running mem0 at full scale requires resetting and re-populating its index per question (approximately 30 s of Vertex AI calls per question). Reranker numbers, in contrast, are now reported on the full 500 (Table 4) using GPU acceleration (GTX 1060 6 GB, 67 min), and the subset-12 result generalizes cleanly to scale (P@5 0.917 $\rightarrow$ 0.920, MRR 0.837 $\rightarrow$ 0.855).

Cross-domain anchor profiles need expert review. The legal, medical, and finance starter anchor sets we ship are written by software engineers reasoning about each domain's typical patterns, not by domain experts. Production deployment in any of these sensitive areas should involve review by qualified domain practitioners before relying on drift detection signals.

Daemon caches a single anchor profile. The current implementation cold-loads one anchor profile at daemon startup; switching profile requires daemon restart. Multi-profile in-memory caching with per-request profile selection is planned for v0.8.

Windows m3 cold-load failure modes. On Windows with Python 3.14, the Hugging Face Hub client (`httpx` backend) intermittently fails to download large models like bge-m3 (2.27 GB), showing zero-byte progress. We work around this via the ModelScope mirror but consider it a deployment-environment fragility rather than a fundamental fix. The recommended production setup is WSL2 or a Linux/macOS host.

Future directions. Beyond the items above, we are particularly interested in: (a) cross-anchor similarity analysis to detect anchor redundancy and over-coverage, (b) integration with white-box methods when activation access is available, (c) automated anchor generation from user history (current implementation is a regex-pattern v0.1; LLM-based v0.2 is on the roadmap), and (d) cross-language transfer of anchor sets via multilingual embedders.

7 Open Source and Reproducibility

7.1 Repository

All code, anchor sets, evaluation scripts, and benchmark results are released under MIT license (anchor files released under CC0 1.0 to encourage forking and adaptation) at https://github.com/chunxiaoxx/nautilus-compass. The repository includes: the runtime daemon (`daemon.py`), hook entry points (`recall.py`, `mid_session_hook.py`, `stop_hook.py`), CLI tools (`feedback.py`, `anchor_generator.py`), six independent evaluation scripts (`tests/eval_*.py`), three domain anchor profiles (`anchors_{legal,medical,finance}.json`), and full documentation (CHANGELOG, RESULTS, CONTRIBUTING, OPEN_SOURCE_READINESS).

7.2 Reproducibility

All numerical claims in this paper are reproducible from the current `main` branch with a single command:

```
git clone https://github.com/chunxiaoxx/nautilus-compass
cd nautilus-compass
pip install -e .[modelscope]
bash tests/run_all.sh
```

The runner produces self-contained log files in `.cache/eval-<timestamp>-<embedder>/` for each evaluation. Total runtime is approximately 90 minutes on a CPU-only host (dominated by m3 cold load and full LongMemEval-S subset 12 run). A subset 4 quick mode runs in under 10 minutes.

The mem0 head-to-head requires a Google Cloud service account JSON for the Vertex AI embedder; see `tests/eval_mem0_headhead.py` for environment-variable setup. The numbers reported in Section 4.4 were produced under these settings.

7.3 Continuous Integration

A GitHub Actions workflow (`.github/workflows/ci.yml`) runs on Ubuntu and macOS, Python 3.10 and 3.12, on every push to `main`. The workflow installs dependencies, runs `selftest.py` for hook plumbing sanity, runs the drift evaluation with bge-small-zh-v1.5 (lighter for CI), and asserts ROC AUC ≥ 0.65 as a regression gate.

7.4 Acknowledgments

This work was deeply influenced by the Persona Vectors line of research at Anthropic (Chen et al., 2025). We thank the authors for clearly articulating the problem of persona drift and for open-sourcing their methodology, which we hope our black-box analog complements rather than competes with. We also thank the BAAI team for the BGE family of multilingual embedders and rerankers (Chen et al., 2024; BAAI, 2024), and the LongMemEval authors (Wu et al., 2024) for releasing a reusable benchmark.

nautilus-compass is developed in the context of running production AI agent deployments for the Nautilus multi-agent platform. The methodology presented here was iterated on real session traces from these deployments before being abstracted into the open-source release.

References

A-MEM Authors. A-MEM: Agentic memory for LLM agents. https://arxiv.org/abs/2502.12110, 2025. arXiv preprint arXiv:2502.12110.

Anonymous. Textdefendr: Empirical robustness of black-box llm safety filters, 2024. ARR submission.

Anthropic. Constitutional classifiers · safety scoring for claude. https://www.anthropic.com/research/constitutional-classifiers, 2024.

BAAI. BGE reranker v2-m3: A lightweight reranker for multilingual information retrieval. https://huggingface.co/BAAI/bge-reranker-v2-m3, 2024. Model card; companion to M3-Embedding (arXiv:2402.03216); accessed 2026-05-07.

Yuntao Bai, Saurav Kadavath, Sandipan Kundu, Amanda Askell, et al. Constitutional ai: Harmlessness from ai feedback. *arXiv preprint arXiv:2212.08073*, 2022.

Jianlv Chen, Shitao Xiao, Peitian Zhang, Kun Luo, Defu Lian, and Zheng Liu. M3-embedding: Multilinguality, multi-functionality, multi-granularity text embeddings through self-knowledge distillation. *arXiv preprint arXiv:2402.03216*, 2024.

Runjin Chen, Andy Arditi, Henry Sleight, Owain Evans, and Jack Lindsey. Persona vectors: Monitoring and controlling character traits in language models. *arXiv preprint arXiv:2507.21509*, 2025. URL https://arxiv.org/abs/2507.21509.

DPT-Agent Authors. DPT-Agent: Distillation path-triggered agent for memory-augmented reasoning. https://arxiv.org/abs/2502.11882, 2025. arXiv preprint arXiv:2502.11882.

Laura Hanu and Unitary. Detoxify: Toxic comment classification with multilingual transformers. https://github.com/unitaryai/detoxify, 2020.

Chris Yuhao Liu, Liang Zeng, Jiacai Liu, et al. Skywork-reward: Reward modeling at scale for practical use. *arXiv preprint arXiv:2410.18451*, 2024.

Mem0 Team. Mem0: Building production-ready ai agents with scalable long-term memory. https://github.com/mem0ai/mem0, 2025. arXiv preprint arXiv:2504.19413; accessed 2026-05-07.

OpenAI. Moderation api · classification of unsafe content. https://platform.openai.com/docs/guides/moderation, 2023.

Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G Patil, Ion Stoica, and Joseph E Gonzalez. Memgpt: Towards llms as operating systems. *arXiv preprint arXiv:2310.08560*, 2023.

Fábio Perez and Ian Ribeiro. Ignore previous prompt: Attack techniques for language models. In *NeurIPS Workshop on ML Safety*, 2023.

Preston Rasmussen, Pavlo Paliychuk, Travis Beauvais, Jack Ryan, and Daniel Chalef. Zep: A temporal knowledge graph architecture for agent memory. *arXiv preprint arXiv:2501.13956*, 2025.

Nils Reimers and Iryna Gurevych. Sentence-bert: Sentence embeddings using siamese bert-networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing*, 2019.

Zhilin Wang, Alexander Bukharin, Olivier Delalleau, et al. Helpsteer 2-preference: Complementing ratings with preferences. *arXiv preprint arXiv:2410.01257*, 2024.

Di Wu, Hongwei Wang, Wenhao Yu, Yuwei Zhang, Kai-Wei Chang, and Dong Yu. Longmemeval: Benchmarking chat assistants on long-term interactive memory. *arXiv preprint arXiv:2410.10813*, 2024.